An Interactive Method to Improve Crowdsourced Annotations

Shixia Liu, Changjian Chen, Yafeng Lu, Fangxin Ouyang, Bin Wang

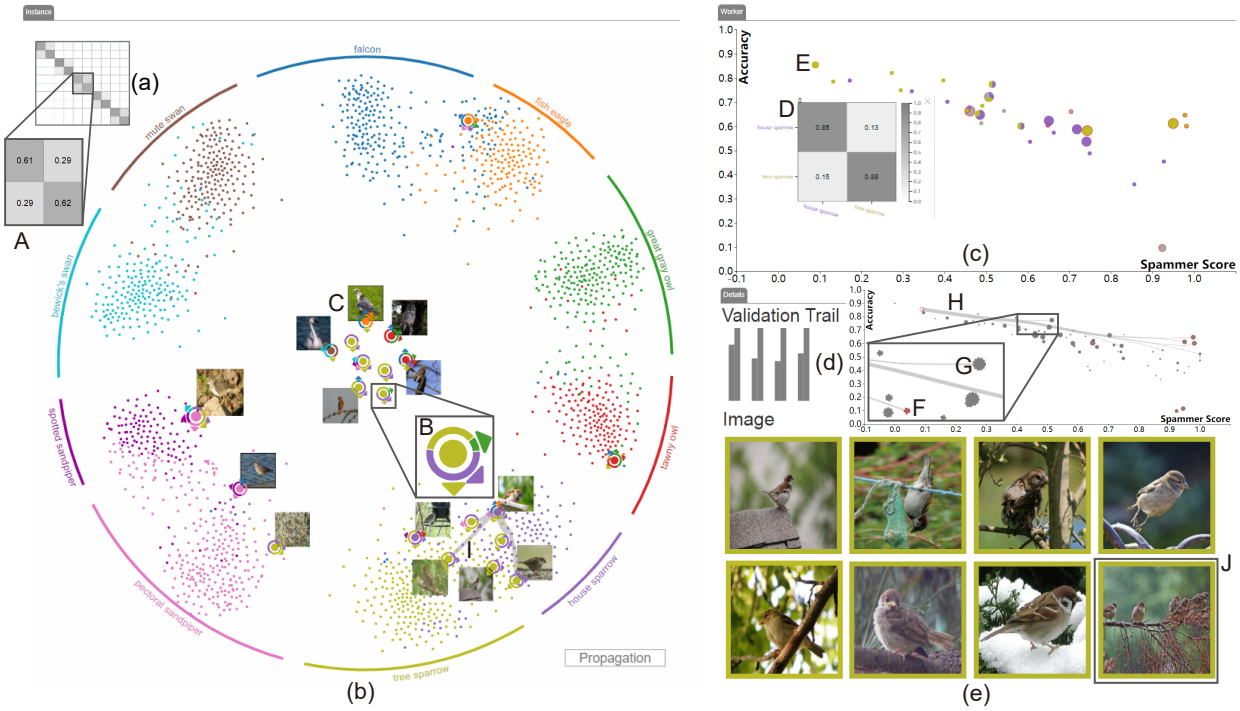

Fig. 1. LabelInspect: (a) the confusion visualization to reveal the confusion degree between different classes; (b) the instance visualization to illustrate the uncertain labels in context; (c) the worker visualization to demonstrate worker reliability; (d) the validation trail to display the number of validated and influenced instances at each validation step; (e) images.

Abstract— In order to effectively infer correct labels from noisy crowdsourced annotations, learning-from-crowds models have introduced expert validation. However, little research has been done on facilitating the validation procedure. In this paper, we propose an interactive method to assist experts in verifying uncertain instance labels and unreliable workers. Given the instance labels and worker reliability inferred from a learning-from-crowds model, candidate instances and workers are selected for expert validation. The influence of verified results is propagated to relevant instances and workers through the learning-from-crowds model. To facilitate the validation of annotations, we have developed a confusion visualization to indicate the confusing classes for further exploration, a constrained projection method to show the uncertain labels in context, and a scatter-plot-based visualization to illustrate worker reliability. The three visualizations are tightly integrated with the learning-from-crowds model to provide an iterative and progressive environment for data validation. Two case studies were conducted that demonstrate our approach offers an efficient method for validating and improving crowdsourced annotations.

Index Terms—Crowdsourcing, learning-from-crowds, interactive visualization, focus + context

1 Introduction

The quality of training data has proven to be a critical factor for the success of supervised and semi-supervised learning [36, 38, 43, 57]. However, labeling a large dataset is expensive and demanding [54]. Therefore, researchers resort to crowdsourcing, which distributes micro labeling tasks to crowds, in order to acquire labeled data [36, 57]. This mechanism, while being efficient and cost-effective, often falls short of high quality results when the task is complicated and requires proven skills or specialties. As a result, crowdsourced annotations may be noisy and poor in quality and usually require additional validation [58]. Correspondingly, there have been some initial efforts to introduce additional expert labels into learning-from-crowds algorithms [27, 36].

Although the performance of the aforementioned methods represents the state-of-the-art for improving crowdsourced labels, they use trusted labels without considering the effort and cost of acquiring them. Data scientists (expert labelers) often have to spend significant time validating the data for better model accuracy, especially when judging the reliability of a worker or evaluating which verifications can bring in a relatively large gain. Moreover, for objects that are difficult to distinguish, data scientists need to consult domain experts who are familiar with the field. Generally, difficulties arise from: (1) Two categories of highly similar-looking objects, for example, the Irish Wolfhound

- *S. Liu, C. Chen, F. Ouyang, and B. Wang are with School of Software, Tsinghua University. Email: {shixia, wangbins}@tsinghua.edu.cn, {ccj17, oyfx15}@mails.tsinghua.edu.cn.*
- *Y. Lu is with Arizona State University. Email: Lu.Yafeng@asu.edu.*

Manuscript received xx xxx. 201x; accepted xx xxx. 201x. Date of Publication xx xxx. 201x; date of current version xx xxx. 201x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.201x.xxxxxxx

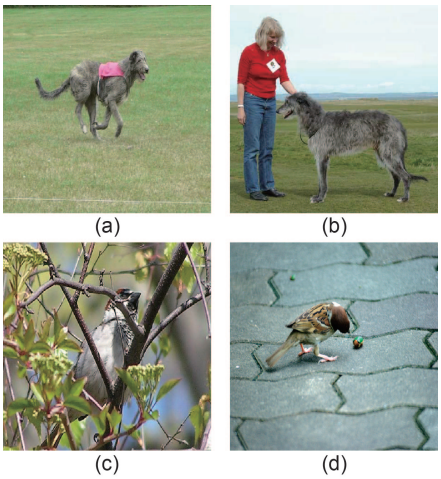

Fig. 2. Objects that are hard to distinguish: (a) Irish Wolfhound; (b)Scottish Deerhound; (c) house sparrow; (d) tree sparrow.

and Scottish Deerhound in Figs. 2(a) and 2(b); (2) the key features for differentiating various objects are partially hidden, for example, in Figs. 2(c) and 2(d), the key features to distinguish these two types of sparrows are the colors of the crown and cheek, part of which is hidden in these two images; (3) some noisy images, which do not belong in the categories of interest, are mixed into the dataset, for example, an image with a Spanish sparrow is contained in the dataset of the house sparrow and tree sparrow. To support an efficient and effective verification, it is necessary to better illustrate the annotation results and minimize the workload of both data scientists and domain experts. However, little work has been done to explore a method of acquiring expert validation efficiently and how to take advantage of such feedback for further annotation improvement.

To facilitate the validation of crowdsourced data we have developed an interactive visual analytics tool, LabelInspect, to improve the annotated results from crowdsourcing. A demo of the prototype is available at `http://visgroup.thss.tsinghua.edu.cn/label`.

LabelInspect combines a learning-from-crowds model [36] with interactive visualizations to provide experts quick access to the most uncertain instance labels and unreliable workers for verification. Expert verifications of instance labels and spammers mutually influence each other by the mutual reinforcement graph model [18, 37], and are propagated to other instances and workers using the learning-from-crowds model [36]. LabelInspect includes a confusion visualization, an instance visualization, and a worker visualization as its main visualizations (Fig. 1). The confusion visualization employs a confusion matrix to measure the confusion rate between classes. In the instance visualization, a novel constrained t-SNE projection is proposed in an attempt to show the uncertain labels in the context of other relevant instances and reveal their influence on each other based on similarities. In the worker visualization, which is utilized to illustrate the reliability of the workers, we propose a class-level spammer score calculation algorithm and combine it with a scatter plot. The three visualizations are tightly coordinated through the underlying models to reflect the validation effects on the relevant instances and workers.

In summary, the main contributions of our work are:

- **A visual analytics tool** that allows data scientists to verify crowdsourced instance labels and unreliable workers in a convenient and fast manner.
- **An iterative and progressive verification procedure** that propagates the influence of expert validations on both instance labels and worker reliabilities to recommend the most informative ones for further verification. Results have demonstrated a large performance improvement.
- **A novel instance visualization** based on a well-formulated constrained t-SNE, **a worker visualization** conveying both global and partial spammer behaviors, and **a confusion visualization**. These visualizations work together to effectively identify the most informative instances and most unreliable workers, and illustrate the influence of each verification on other instances and workers.

2 Related Work

Our work is related to integrating machine learning techniques into visual analytics [14, 20, 38, 39, 42, 55, 63], in particular, combining interactive visualization with learning techniques to improve data quality. In the field of visualization, many methods have been proposed to obtain high-quality data. Most existing relevant work can be classified into three branches: visual data processing, visual data retrieval, and interactive labeling.

Visual data processing. To support data processing, Kandel et al. [30] proposed Wrangler, which is a visual analytics system focusing on creating data transformations. Maciejewski et al. [47] employed the Box-Cox transformation to transform data into the best normalized distribution for effective visualization and analysis. Other than data transformation, some methods have focused on anomaly detection and outlier removal. For example, the system developed by Kandel et al., Profiler [31], coupled automated anomaly detection with linked summary visualizations and interactive data exploration to help analysts assess data quality. Hao et al. [22] supported noise removal prior to building prediction models for seasonal time series data. Wilkinson [61] developed a distributed algorithm for detecting outliers in big data and visually illustrated the outliers for further inspection. In terms of crowdsourced data, Willett et al. [62] introduced a crowd-assisted clustering method to identify and consolidate redundant responses. Park et al. [50] developed a visual analytics platform to analyze medical crowdsourced data. In contrast to the above visual data processing techniques, LabelInspect considers the relationship between worker reliability and instance label uncertainty, and integrates the mutual reinforcement graph model and learning-from-crowds model to propagate expert verifications in different phases.

Visual data retrieval. Data retrieval is an effective way to obtain data of interest for data-related applications. Several methods have been proposed for visual data retrieval. For example, Krause et al. [34] presented a visual analytics system, COQUITO, which supports cohort construction by iteratively updating queries through a visual interface. Liu et al. [37] developed a microblog retrieval system, MutualRanker, which considers posts, users, and hashtags together as integral for retrieving and ranking data. It presented the results on a graph visualization along with its uncertainty calculated from a mutual reinforcement graph model. Lu et al. [45] utilized semantic similarity to obtain relevant textual records based on an interactively built concept map on a force-directed layout. Wall et al. [59] developed Podium to interactively rank and retrieve multi-variate data based on users' subjective preferences, which are obtained by capturing users' drag interactions on a table view. All the aforementioned methods aim to acquire relevant data without consideration of labels, which thus cannot be directly extended to improve the quality of labeled data. Therefore, we tightly integrate visual analytics with the learning-from-crowds model to provide an iterative and progressive environment for verifying uncertain instance labels and unreliable workers.

Interactive labeling. Many visualization researchers have also proposed employing active or incremental learning to help users interactively label a variety of data. Heimerl et al. [23] presented a method that incorporates active learning and interactive labeling for document classification. Paiva et al. [49] supported interactive labeling by helping users interpret misclassified instances on a Neighbor Joining tree and a similarity layout view. This concept of interactively labeling data as part of a model learning process is also supported by other visualization works [4, 6, 7, 9, 25]. An experimental study conducted by Bernard et al. [5] demonstrated the promise of visual-interactive labeling. It showed that visual-interactive labeling usually outperforms active learning when class distributions are well separated by dimension reduction. Different from these methods, our work focuses on labeling crowdsourced data, which has specific data quality issues. The labeler not only needs to label instances but also analyze worker behavior. In addition, s/he often needs to understand the influence within instance

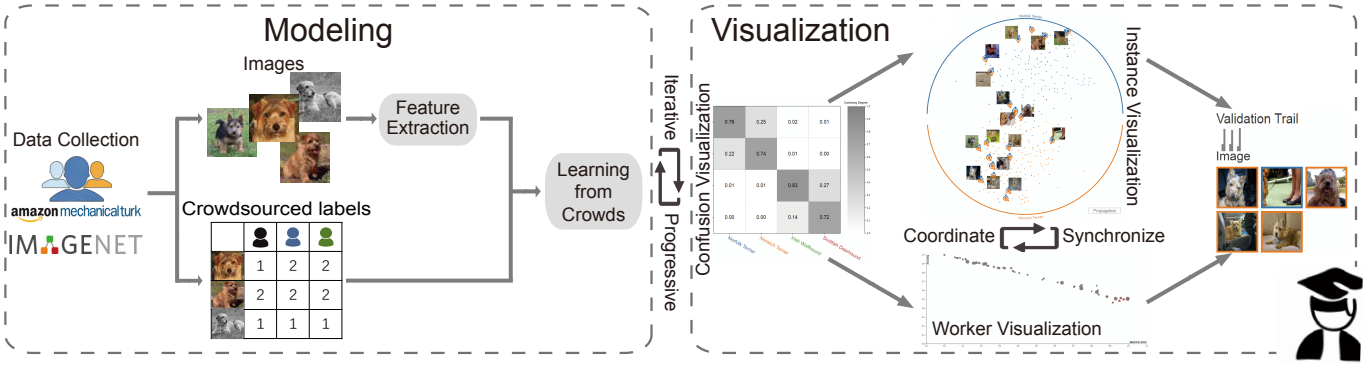

Fig. 3. LabelInspect overview: the data modeling module estimates correct labels from crowdsourced annotations and updates the labeling results based on expert validation; the visualization module allows analysts to interactively validate instance labels and spammer workers.

labels or workers as well as the mutual influence between them. Therefore, LabelInspect consists of three highly coordinated visualizations to facilitate the labeling process.

In the field of machine learning, researchers have introduced expert validation into learning-from-crowds models [27, 36]. These methods have demonstrated the effectiveness of leveraging expert labels to reduce the labor involved and improve the performance. However, they all assume expert labels can be acquired easily. This is not true in many real-world applications, in which experts have to repetitively browse the data and carefully compare crowdsourced labels to provide useful validations. Consequently, developing an effective validation procedure is of theoretical and practical significance.

3 Design of LabelInspect

In this section, we introduce the task analysis process and collected tasks, as well as the visual analytics tool built on these tasks.

3.1 Task Analysis

This research is motivated by our own research projects that employ supervised and semi-supervised learning as an analysis method. Because data quality is critical for data-driven tasks [1], in these projects, after collecting labeled data from Amazon Mechanical Turk (AMT), we often spent a lot of time and effort cleaning the data, including discovering the most problematic instance labels and spammers. This process is tedious and requires domain expertise in complex cases.

In order to identify the main scenarios in the aforementioned validation process, we conducted three interviews with four data scientists. Two of them are R&D researchers from Inventec, whose job responsibility is to validate the "pass/fail" labels of boards along the product line (A_1, A_2). The other two are senior PhD students who often collect and validate image labels for crowdsourcing-related research (A_3, A_4). For brevity's sake, we refer to data scientists as analysts and to domain experts as experts throughout the rest of the paper. The interviews were semi-structured with a focus on probing the participants validation needs and processes by asking the following questions.

- How do you improve the quality of crowdsourced annotations?
- What are the major steps in the validation process?
- What type of information helps the validation process?

In the first interview, the analysts said that they needed to understand the crowdsourced labels and the labeling behavior of workers from different perspectives, so that they can quickly identify the most problematic data and suspicious workers. For objects that are easily confused with each other, they usually consult with a domain expert who knows the data. In such cases, an intuitive visualization is preferred to help explain the reason for the confusion to the domain expert within the proper context. Based on the interviews and our previous research experience, we identified the following tasks:

T1 - Inspect the confusion degree among different categories and select the most confusing ones for detailed examination. This is especially important when multiple-category labeled data is considered.

T2 - Understand the instance distribution under different category labels, examine the confused instances between categories in context, and then quickly identify the most problematic labels.

T3 - Analyze the reliability of each worker, form an assumption of the spammer degree of the worker, verify the formed assumption by checking the corresponding annotation results, and recognize spammers in order to improve the labeling accuracy.

T4 - Check the influence within instances/workers, as well as mutual influence between them. If the label of an instance is modified, the analyst wants to understand its influence on other instances and the workers who annotated this instance, and vice versa.

T5 - Examine the validation trail to understand the influence of each validation on other instances/workers. The validation trail displays the number of instances validated by the analyst in each propagation round and the number of instances whose label estimates have been changed with these verifications. By checking the trail, an analyst is able to decide when to stop a validation process or roll back to a previous state. For example, if the recent influence is very small, it might be one indication to stop the validation process.

3.2 System Overview

Guided by the above tasks, we designed a visual analytics tool, LabelInspect, to help analysts validate crowdsourced labels and acquire high-quality training data. It consists of two modules: data modeling and interactive visualization. They work together to provide an iterative and progressive validation solution for data validation.

As shown in Fig. 3, the data modeling module focuses on collecting the crowdsourced annotations and estimating correct labels from noisy crowdsourced data using a learning-from-crowds model. In our implementation, we take image data as an example to illustrate the basic idea of the proposed method. We collected raw data from ImageNet and obtained the crowdsourced labels from AMT. The estimated labels were derived using the max-margin-majority-voting (M^3V)-based model [36]. After several verifications from the analyst are fed back, the M^3V-based model incrementally updates the labeling results to start a new round of validation. The visualization module consists of three coordinated visualizations to enable analysts to interactively validate crowdsourced data and improve the labeling accuracy. By examining the confusion between different classes in the confusion visualization, the analyst can pick out the most confused ones to start the validation process (**T1**). Then the instance visualization is employed to show the uncertain labels in the context of other relevant instances (**T2**). The experts first verify several seed instances and/or workers, and then candidate instances and workers are selected for further validation. The influence of verified results is propagated to relevant instances and workers through the M^3V-based model (**T4**). In the worker visualization, a scatter plot is employed to illustrate worker reliability (**T3**). By checking the spammer score and labeling results, the analyst can identify the highly reliable workers and spammers. The validation of workers will also propagate to other data via the M^3V-based model. The three visualizations are closely coordinated through a set of analy-

sis methods, such as candidate selection for further verifications (**T4**), validation propagation to reflect the validation impact on the relevant instances/workers (**T4**), and exploration of the validation trail to roll back to a previous validation step (**T5**).

4 Data Modeling

The current implementation uses two image datasets, a dog dataset with 4 categories and a bird dataset with 10 categories from ImageNet [16]. The dog dataset already contains the crowdsourced annotations [65]. For the bird dataset, we utilize AMT to collect crowdsourced annotations. Each worker labels an image at most once, and each image is labeled 12 times. We exclude workers with a low number of annotations (less than 60 labels) or low accuracy (less than 20% on the gold standard images). The resulting crowdsourced dataset is available at `http://cgcad.thss.tsinghua.edu.cn/shixia/crowd.zip`. For each dataset, we first perform feature extraction, then employ a learning-from-crowds model to infer the initial true labels and incrementally update the labeling results based on validation.

Feature Extraction. Previous studies have shown that feature vectors extracted from deep convolutional neural networks (CNNs) trained on a large labeled dataset, such as the ImageNet ILSVRC 2012 dataset [51], can represent the image very well and be used in different pattern recognition tasks [17, 64]. As a result, for the dog dataset, whose categories are all covered by the ILSVRC 2012 dataset [51]), we employ the outputs of the last but one fully-connected layer of a pre-trained VGG-16 [52] as the feature vectors. In particular, we utilize the pre-trained Keras VGG-16 model on the ILSVRC 2012 dataset [12]). The aforementioned feature vectors work well for the dog dataset since all 4 categories in this dataset are included in the ILSVRC 2012 dataset.

However, the bird dataset includes images of several species that are not contained in the ILSVRC 2012 dataset, thus the extracted features with the Keras model do not differentiate them well. To solve this problem, we employ a fine-tuning method [21] to adapt the pre-trained VGG-16 model to our task. In particular, we replace all the fully connected layers of the Keras VGG-16 model with three fully connected layers and update the model weights with a set of additional training samples. This sample set consists of 800 images whose labels are inferred by the learning-from-crowds model with the least uncertainty and 10,672 images of the 10 bird species from ImageNet, resulting in a training set of 8,030 images and a validating set of 3,442 images. We also use the output of the last but one fully-connected layer of the fine-tuned model as the feature vector of each image.

The Learning-From-Crowds Model. We employ a semi-supervised learning-from-crowds method, the M^3V-based model [36], to estimate the true labels from noisy crowdsourced annotations and gradually integrate the expert labels, which are confirmed by the analyst via a step-by-step validation process. The detailed formulation and solution of this model can be found in [36]. The advantage of using this method is that it introduces additional expert labels into the learning-from-crowds framework and incrementally updates the learning model as new expert labels arrive. This method is also utilized to infer the initial true labels from the crowdsourced annotations.

5 Visualization

LabelInspect consists of three major visualizations. The confusion visualization illustrates the confusion degree among classes and helps select a few classes to start (**T1**). The instance visualization uses a constrained t-SNE projection to visualize the instances of interest and highlights the most informative ones using uncertain glyphs (**T2**). A scatter plot is employed in the worker visualization to demonstrate worker reliability (**T3**). Instance and worker visualizations are coordinated so that verifications on either visualization will update both of them to reflect their mutual influence (**T4**). In addition, a validation trail and clear images are provided to aid the validation process (**T5**).

5.1 Confusion Visualization

The exploration of confusion matrix has been supported in Talbot et al.'s EnsembleMatrix [55] to understand and compare multiple classifiers. In our approach the concept of confusion matrix is extended and used to explore the relationship between worker labeling and model estimates. The confusion visualization (Fig. 1(a)) is developed to illustrate whether workers confuse two or more classes. This is important because in a multi-class labeling task, identifying a few (often two) mixed classes can dramatically reduce the workload and difficulty of validating the annotations. In order to effectively visualize possible confusion patterns, we calculate a worker confusion matrix and reorder it to reveal block patterns where each block can be considered to be one subset of the confusing classes.

The worker confusion matrix represents the mismatch between the workers' labeling results and model-inferred results. It is an $m \times m$ matrix where m is the number of classes. Each column represents the inferred class from the M^3V-based model, and each row represents the label assigned by the workers. The cell in the ith row and jth column represents the percentage of times that instances inferred to be the column class c_j have been labeled as the row class c_i. We denote the percentage as $LabelP(c_i, c_j)$. For example, in Fig. 1A, the model-inferred house sparrows are labeled as " house sparrow" 61% of the time, and as "tree sparrow" 29%. Generally, $LabelP(c_i, c_j)$ and $LabelP(c_j, c_i)$ are not equal, and we take their average to be the similarity between c_i and c_j.

Once the worker confusion matrix has been calculated, a matrix reordering algorithm is developed to reveal the block pattern visually. A reordered matrix with a block pattern shows coherent rectangular areas (blocks) and each block indicates the set of confusion classes. As the Optimal-Leaf-Ordering (OLO) algorithm [2] has been shown to be optimal for demonstrating blocks [3], we have employed this reordering algorithm.

Our confusion visualization uses a single hue sequential color scheme to encode each cell according to its label percentage value so that the groups are easily identified as dark blocks. A quantitative color scheme is used to distinguish classes that are labeled along the left and bottom sides of the matrix. This visualization supports class selection for the instance inspection, and the percentage values update in each validation round to cue the analyst on the validation progress.

5.2 Instance Visualization

Once we have the classes selected, the instance visualization will show all the instances inferred to be in these classes and the analysts can explore the instance labels and work on instance-level validation.

Visual Encoding. This visualization is designed as a circular-based constraint layout (Fig. 1(b)), where the outer arcs represent classes and the length is proportional to the number of instances. Instances are plotted within the circle. Same as the confusion visualization, the color represents the class.

Our underlying model will recommend some uncertain instances that are encoded using uncertain glyphs, while the remaining instances are encoded as simple dots with colors indicating their inferred classes. The uncertain glyph, as shown in Fig. 1B, consists of a centered dot indicating the inferred class, the surrounding colored arcs denoting the distribution of worker assignments, arrowhead markers pointing towards corresponding class arcs (marked by the workers) on the outer circle, and an optional snapshot providing a quick spot of the instance (Fig. 1C). In our implementation, 20 uncertain instances are selected by the MRG model (Sec. 5.4). The position of an instance is determined by a constrained t-SNE projection, which is explained later in this section. The snapshot position is calculated using a clutter-aware label-layout algorithm [48] to ensure that the snapshot is close to its instance while not cluttered with other snapshots. Given these snapshots, analysts can quickly judge whether the prediction of the model is accurate.

Constrained t-SNE Projection. We propose a constrained t-SNE projection for instance visualization so that 1) similar instances are grouped together; 2) instances tend to plot towards their inferred class and the most uncertain ones are pulled by all its possible class labels; and 3) instances try to keep their previous positions when being updated.

Although t-SNE [46] maintains the relationships between instances in the low-dimensional space, it does not consider the relationships between the crowdsourced labels and the inferred labels. As a result,

it is difficult to utilize t-SNE to clearly reveal label errors. A supervised t-SNE is used in UTOPIAN [13] where the distances between the same class instances are decreased by a parameter. In addition, TopicLens [33] proposed a guided t-SNE where the group centroids on the projected 2D space are first calculated with a small set of instances and then the centroids are used to add constraints for additional data points. These methods consider instance labels or projection stability to some extent. However, they do not control the layout position of the groups and cannot meet our need that certain instances are close to its class arc while uncertain instances are projected to the center. In addition to the class-level readability supported by the guided t-SNE, we also want to maintain instance-level stability so that the same instance will be close to its previous position after each update.

Therefore, we have developed a novel constrained t-SNE projection that balances readability and stability for the instance layout. Readability helps preserve neighborhoods and clusters (overall readability), reveals class association, and accentuates abnormal labels (class readability). Stability helps control the position of the instance in the iterative process and provides a stable context. Thus, for m classes and n instances, we minimize the following cost function,

$$f_{cost} = \alpha \cdot KL(P \parallel Q) + \beta \cdot KL(P_c \parallel Q_c) + (1-\alpha-\beta) \cdot KL(P_s \parallel Q_s), \quad (1)$$

where $KL(\cdot \parallel \cdot)$ is the Kullback-Leibler (KL)-divergence between two distributions. The first KL-divergence is the same as the original t-SNE [46] for projecting the n instances and preserving the overall readability. The second KL-divergence is to maintain the class readability. The last KL-divergence is to improve stability in the iterative validation process. $\alpha, \beta \in [0,1]$ are two weights to balance them.

We introduce m class constraint arcs, one for each class, and n virtual stability constraint points, one for each instance. P_c, $Q_c \in \mathbb{R}^{n \times m}$ are the joint probability distributions that measure pairwise similarities between the instances and constraint arcs in the high-dimensional space and low-dimensional space, respectively.

$$(P_c)_{ij} = \begin{cases} \exp(-d_{ij}^h/2\sigma^2) \cdot \mathbb{I}(c_j \in \mathbb{L}_i) & e_i \in \mathbb{U} \\ \exp(-d_{ij}^h/2\sigma^2) \cdot \mathbb{I}(c_j = l_i) & e_i \notin \mathbb{U} \end{cases}, \quad (2)$$

$$(Q_c)_{ij} = (1+ \parallel d_{ij}^p \parallel^2)^{-1}, \quad (3)$$

where $\mathbb{I}(\cdot)$ is an indicator function, d_{ij}^h is an estimated distance between instance e_i and arc of class c_j in the high-dimensional space, and d_{ij}^p is the distance between instance e_i and the nearest point on the arc of class c_j in the projected space. l_i represents the inferred label of instance e_i, $\mathbb{L}_i$ is the set of labels that instance e_i has been assigned by the crowd workers, and $\mathbb{U}$ is a set of uncertain instances. To estimate d_{ij}^h, we build one hypersphere in the high-dimensional space for each class. This hypersphere uses the class's barycenter as its centroid and the distance from the centroid to the farthest point as its radius. Then d_{ij}^h is defined as the shortest distance from instance e_i to the hypersphere.

Given the way we define P_c, uncertain instances will have multiple class constraints decided by the labels assigned by the workers. These constraints add the tendency to project such instances to the middle/center of their confusion classes. For other instances, instead, only one class constraint from the inferred class will be added, and they will be pulled towards their inferred class arc.

For the stability constraint, the virtual point takes the same position of the instance in the high-dimensional space and the last projected position of the instance as its 2D position. P_s, $Q_s \in \mathbb{R}^{n \times 1}$ are the joint probability distributions that measure pairwise similarities between the instances and stability constraint points in the high-dimensional space and low-dimensional space, respectively. $(P_s)_i \in [0,1]$ represents the strength of the stability constraint added to instance e_i. In our implementation, P_s changes after every propagation.

$$(P_s)_i = \begin{cases} 0 & e_i \text{ is validated} \\ 0.5 & e_i \text{ is not validated, but its label has changed} \\ 1 & \text{otherwise} \end{cases}, \quad (4)$$

Stability constraints for validated instances are set to be 0 so that the instances are able to be freely attracted to the confirmed class. We set a middle value, 0.5, for the instances whose inferred labels have changed because of the propagation. In this way, their positions tend to be closer to the arc of its newly inferred class but not too far away from its previous position for easy tracking. Other instances stick to their previous positions. Q_s is the similarity in the 2D space between instances and their previous positions. We use $\boldsymbol{y}_i'$ to denote the last projected position of instance e_i and Q_s is defined as: $(Q_s)_i = (1+ \parallel \boldsymbol{y}_i - \boldsymbol{y}_i' \parallel^2)^{-1}$.

Flow of Influence. Once an instance label is verified, the MRG model will assign a high prior saliency to this instance and re-rank all instances to reselect the 20 uncertain instances apart from the validated ones. This new set of uncertain instances are displayed using instance glyphs. In addition, the influence flow from the verified instance to the 5 most influenced candidate instances is displayed as a flow map, as illustrated in Fig. 1I. The flow map helps to expose the internal status of the validation propagation and inform the analyst of the effect. The layout of the flow paths from one source to multiple targets is based on the flow map layout [10] and the edge bundling developed by Holten et al. [26]. The width of the flow between two instances is proportional to their similarity value, which denotes the influence degree from the verified instance to the other instances on the flow.

5.3 Worker Visualization

The inference of instance labels and worker reliability mutually influence each other in crowdsourced data validation. Therefore, we have developed a worker visualization to illustrate worker reliability and enabled the validation of crowdsourced data from the worker-level data.

Overview. The worker visualization only plots workers who have labeled the instances associated with the selected confusion classes. A scatter-plot is implemented where the x-axis is the spammer score, the y-axis is the accuracy according to model inferred labels. One dot represents one worker where the size is proportional to the number of instances s/he has labeled. The accuracy of a worker is measured by the percentage of his labeled instances that match the model predictions. The spammer score is calculated by an approach that combines the spammer detection method proposed by Hung et al. [27] and block detection to support finding partial confusing spammers, which is introduced below. Similar to uncertain instances, the MRG model recommends K candidate workers and highlights them on the scatter plot after a spammer is confirmed. In our implementation (Fig. 1(c)), $K = 8$, and these highlighted dots all have high spammer scores and low accuracy.

An individual confusion matrix is coordinated with the scatter plot to reveal the worker's labeling performance (Fig. 1D). The design of this matrix is the same as that of the worker confusion matrix except that the individual confusion matrix only analyzes the data for one worker on the selected confusion classes. The coordination between the scatter plot and the individual confusion matrix is useful for revealing patterns on a worker's labeling performance. For example, the matrices shown in Fig. 4 indicate two different confusion patterns. Fig. 4(a) indicates that the worker has a problem distinguishing house sparrows from tree sparrows, but is able to label falcons and fish eagles pretty well; while Fig. 4(b) shows the opposite pattern for another worker. As such, the

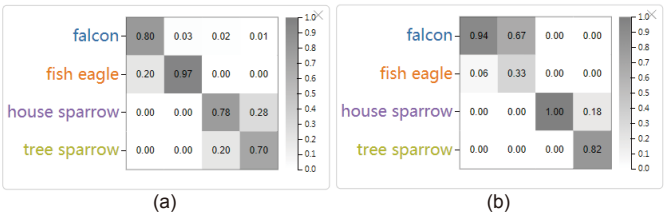

Fig. 4. Two different confusion patterns: (a) the worker has a problem distinguishing house sparrows from tree sparrows, but has a high labeling accuracy on the classes "falcon" and "fish eagle"; (b) the opposite pattern for another worker.

analyst can verify for which classes a worker is reliable or unreliable. Given this verification, the MRG model is able to update candidate instances and workers. A flow map is drawn to highlight the most influenced workers. If a worker is identified as reliable, the M^{3}V-based model will increase the worker's prior reliability score. If a worker is identified as a spammer for a few classes, we remove the most likely incorrect labels assigned by the worker, and then as suggested by the analyst, we lower the prior of the reliability of the worker.

Spammer Score. Hung et al. [27] calculated the spammer score as the distance from the prediction matrix to its closest rank-one approximation for random and uniform spammers. However, for multi-class labeling tasks, a spammer may only confuse between a subset of the classes rather than all of the entries. Comparing the worker's individual confusion matrix to its rank-one approximation cannot detect these partial spammers. To solve this problem, we developed a block-based spammer detection method, which calculates the spammer score in the following two steps:

Step 1: Block Detection is used to identify the block pattern in a worker's confusion matrix. Matrix reordering, as described in Sec. 5.1, is first applied to put blocks, if any, along the diagonal. We denote this reordered matrix as L. In our work, a block indicates that a subset of the classes are difficult to separate by the workers, and therefore, we consider all blocks to be square. Detecting these squared blocks is equivalent to finding segments along the ordered classes such that the sum of the density in each block indexed by each segment is maximized. We denote $B(i,j)$ as the block from class c_i to class c_j in the ordered matrix L. The density of $B(i,j)$ is defined as:

$$density(i,j) = \begin{cases} (\sum_{x=i}^{j}\sum_{y=i}^{j} L[x,y] - \sum_{x=i}^{j} L[x,x])/((j-i)(j-i+1)) & if \quad i<j \\ 0 & if \quad i=j \end{cases} \tag{5}$$

where $L[x,y]$ is the value of the xth row and yth column in L. We remove the diagonal values and consider only the "confusion" part (off-diagonal cells). We solve this following the time series segmentation problem using a dynamic programming approach [24].

Step 2: Aggregation is then preformed to aggregate the spammer score, $s(w)$, for worker w, based on the detected number of blocks, k. The goal is to assign the highest spammer score to one-block workers who mixed all classes, and the lowest spammer score to m-block workers who separated all classes. For middling workers, their score depends on the number of blocks and how confused they are within each block. As such, the spammer score for worker w is defined as: $s(w) = \frac{1}{k}\sum_{i=1}^{k} \min_{\hat{M}_w(i)} \| M_w - \hat{M}_w(i) \|_F$, where M_w is the individual confusion matrix for worker w, and $\hat{M}_w(i)$ is the rank-i approximation matrix to M_w, using the Frobenius norm. This problem is solved using singular value decomposition [19].

Crowd Workers Characterization. Previous studies have characterized crowd workers into five types in terms of their expertise, including reliable workers, normal workers, sloppy workers, uniform spammers, and random spammers [32, 27]. These types are specifically defined for binary classification. For multi-class classification, in addition to these five types, we identify a sixth type, partial spammers, who have certain ability to label a subset of the tasks, but behave as random or uniform spammers for the rest. Fig. 5 illustrates the characteristics of the six types of workers by using the scatter plot and the individual confusion matrix.

- **Reliable workers** have high accuracy and low spammer scores. Their confusion matrices have very high values in the diagonal cells and very low values in the off-diagonal cells (Fig. 5A).
- **Normal workers** have relatively high accuracy and low spammer scores, but not as accurate as reliable workers. This is evidenced by the fact that their matrices have relatively high values in the diagonal cells and relatively low values in the off-diagonal cells (Fig. 5B).
- **Sloppy workers** have low accuracy and medium spammer scores. Their labeling is not very reliable in general, and it is hard to find a high quality subset of task. Accordingly, in the confusion matrices, the values of the diagonal cells are a little higher than those of the off-diagonal cells. However, the difference is trivial (Fig. 5C).

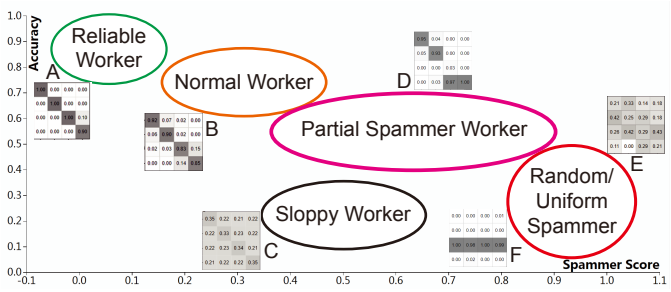

Fig. 5. Illustration of six different worker types on the worker visualization.

- **Partial spammers** have medium accuracy and relatively high spammer scores. Their confusion matrices show obvious block patterns and they either can label some classes accurately or separate instances between class families (e.g.,a sparrow is only mislabeled as another type of sparrows, but not as an eagle or owl (Fig. 5D)).
- **Random spammers** have low accuracy and high spammer score. They often carelessly assign labels to all instances. Thus, the corresponding confusion matrices demonstrate no clear patterns. Typically, the values of the diagonal cells are similar to those of the off-diagonal cells (Fig. 5E).
- **Uniform spammers** also have low accuracy and high spammer scores, similar to the random spammers. These workers usually assign the same label to all instances. As a result, their confusion matrices have an all-one row with other cells being 0 (Fig. 5F).

Random spammers and uniform spammers are considered to be the most unreliable workers. Though they have different patterns on their individual confusion matrices, they are all plotted towards the lower right corner on the scatter plot (Fig. 5).

5.4 Interactive and Iterative Exploration Environment

This section introduces the coordination of the three major visualizations. It includes a MRG model that uses the mutual influencing instance and worker information to select candidates, a detail view to assist instance validation and record historical verifications, and a set of interactions that link these visualizations.

Mutual Reinforcement Graph Model. In order to help analysts validate data efficiently, uncertain instances and workers are selected by the system based on instance label uncertainty and worker unreliability. This candidate selection can be formulated as a ranking problem. A straightforward solution is to leverage the PageRank algorithm [8]. However, the PageRank algorithm only influences items in the same graph, while the MRG model has advantages in terms of leveraging both the relationships within the instances or workers, and the mutual influence between them. As a result, we employ the MRG model to rank the items in the instance graph and worker graph. The relationship between these two graphs indicates which instances have been labeled by a worker. The instance graph has each node represent one instance, and each pair of nodes are connected by an edge whose weight is the similarity between them. The worker graph is built in the same way, while worker similarity is calculated using SimRank [29] on a bipartite instance-worker graph. We build the instance-worker bipartite graph as described in [28]: **(i)** each worker w is represented by a node; **(ii)** each instance e is represented by m nodes (m is the number of classes): $e(c_1)$,$e(c_2)$,...,$e(c_m)$; **(iii)** an edge $(w, e(c_i))$ is added if worker w labels instance e as class c_i.

The MRG model employs a similar method to PageRank [8] to model the mutual influence between workers and instances, as well as the influence within each graph. We define R_e and R_w as the ranking score vectors for instances and workers respectively. The initial scores for all items are the same ($1/n_t$, n_t is the total number of instances and workers), and they are calculated iteratively through the following equation:

$$\begin{bmatrix} R_e \\ R_w \end{bmatrix} = \lambda \begin{bmatrix} \alpha_{ee}M_{ee} & \alpha_{we}M_{we} \\ \alpha_{ew}M_{ew} & \alpha_{ww}M_{ww} \end{bmatrix} \begin{bmatrix} R_e \\ R_w \end{bmatrix} + (1-\lambda) \begin{bmatrix} V_e \\ V_w \end{bmatrix} \tag{6}$$

where M_{xy} denotes the affinity matrix from x to y, and x, y can be instances or workers. α_{xy} is used to make a trade-off of the mutual reinforcement strength among instances and workers. λ is the damping factor for determining the importance of a worker or an instance. V_e and V_w are the prior saliency (uncertainty and spammer score) of instances and workers, respectively.

In the progressive validation process, the ranking scores are updated by increasing the prior saliency of verified instances and workers. Once the ranking scores change, the instance visualization and worker visualization employ the MRG model to update their uncertain candidates.

The Detail View and Interactions. In order to assist instance validation, we have developed a detail view for analysts to inspect instances and check their validation histories. A set of rich interactions are also provided to assist the validation process.

Instance Inspection in Detail. We allow the analyst to check the selected instance(s) in detail to confirm his/her validation. The analyst can click on one candidate instance or select a few using a lasso selection in the instance visualization, and the detail view will display the images associated with these instances. Similarly, the analyst can click on a worker on the scatter plot and this will bring the images labeled by this worker into the detail view. Clicking on a cell of the worker's prediction matrix will further filter down the images to those associated with the selected class. The analyst can enlarge an instance image and crop the image if s/he thinks the feature can be improved by removing some background objects or focus more on a local area. The cropped image will be fed back to the feature extraction step which triggers a linked update of the model, the instance and worker visualizations.

Validation Trail and Roll Back. We have developed a validation trail to show the numbers of validated instances and the instances whose labels are changed by the verifications. Fig. 1(d) shows the validation trail after a few rounds of validation and propagation. Each round is represented by two bars, where the left bar represents the number of verified instances and the right bar indicates the number of impacted instances. Analysts can click on the bars to recall which instances are verified and impacted. These instances will be displayed in the detail view as images. In addition, they are allowed to roll back to a previous validation status by clicking on "redo" under each validation bar.

Coordination between Instance and Worker Validations. A core feature of LabelInspect is its coordination between instance and spammer validation. We support a tightly linked interactive exploration of the instance and worker visualizations. For example, when an instance is hovered over, the workers who labeled this instance will be highlighted. When one or multiple instances are selected, displayed workers will be filtered down to those who have labeled any of these instances, and the dot for each worker will also change to a pie chart. The dot size matches the number of instances the worker labeled among the selected ones and the area of the sector represents the ratio of the class labels the worker assigned to these instances. The color of each sector matches the color of the corresponding class. On the other hand, mousing over a cell of a worker's confusion matrix will highlight all instances labeled by him/her on the instance visualization. Given these linked interactions, the analyst can explore instance labels and worker reliabilities conjointly. Validation on either side will update the uncertain candidates on both instances and workers through the MRG model, and the propagations of analysts' validations will also update both instance label estimates and worker's spammer scores.

6 Case Studies

We conducted two case studies to demonstrate how to effectively use LabelInspect to improve the quality of crowdsourced annotations from both the instance and worker perspectives.

6.1 Bird Image Validation

In this case study, we collaborated with A_1 and A_3 to show how LabelInspect helps improve crowdsourced annotations with lower estimation accuracy. Initially, A_1 indicated that the model estimation accuracy of the bird dataset was relatively low (accuracy: **82.05%**), and wanted to improve it by using LabelInspect.

Overview (T1, T2). To begin with, A_1 looked at the confusion visualization (Fig. 1(a)) and quickly found that among the 10 species, most of the disagreements involved two species in one block. There were 5 pairs of confusion classes in total. Furthermore, the confusion degrees between these 5 pairs were relatively low. To further confirm this observation, A_1 selected all 10 species and turned to the instance visualization. As shown in Fig. 1(b), only a few instances were projected to the center of the view, and the majority of the confusions were within each confusion class pair. Among the 5 pairs, the sparrow pair ("house sparrow" and "tree sparrow"), the sandpiper pair ("pectoral sandpiper" and "spotted sandpiper"), and the swan pair ("bewick's swan" and "mute swan") were confused the most often. Therefore, A_1 decided to start his validation with the sparrow pair (the 'tree sparrow' class in olive and the 'house sparrow' class in purple) since it involved the most confusion.

Starting with Instance Analysis (T2). With "house sparrow" and "tree sparrow" selected, the olive and purple dots were often mixed in the instance visualization. A_1 looked at the snapshots of the uncertain instances recommended by the tool and he found that many of those in the center were misclassified by the M^{3}V-based model. For example, the 5 instances circled in Fig. 6A were estimated to be "tree sparrow" (olive), however, they should have been "house sparrow" (purple). Therefore, A_1 corrected their labels. It was very interesting to find that these 5 house sparrow instances were all misclassified as "tree sparrow" by the model. A_1 wanted to understand why this happened. He then selected these 5 instances using the lasso selection in the instance visualization to examine how the workers labeled them. Given the selected instances, the scatter plot (Fig. 1(c)) in the worker visualization filtered for any workers that had labeled these 5 instances, and each of the remaining workers was then represented as a pie chart displaying the ratio of the labels they assigned to these instances.

Focusing on Spammer Analysis (T3, T4). The worker with the highest accuracy and lowest spammer score (worker A in Fig. 1E) caught A_3's attention because he noticed that 2 instances were mislabeled by this worker though s/he had a high inferred reliability. Worker A's confusion matrix (Fig. 1D) indicated that his/her labeling results mostly matched with the estimates of the model, as the diagonal cells of the matrix had high values but off-diagonal cells had low values. Recall that the 5 mislabeled instances were all recognized as class 'tree sparrow' (olive) by this worker and the model. A_3 hypothesized that this worker and the model must have both mislabeled many house sparrows (purple) as tree sparrows (olive). A_3 then loaded the corresponding images into the detail view. These images were all estimated to be tree sparrow by the model and labeled as "tree sparrow" by worker A. A_3 checked the first 8 images and found 4 of them should have been "house sparrow." He also noticed that one image (Fig. 1J) was confusing because the key features (the colors of crown and cheek) were hidden. A_3 consulted an expert (an ornithologist) and confirmed it to be "house sparrow." This finding verified his hypothesis that worker A and the model both mislabeled many house sparrows as tree sparrows, and worker A's accuracy needed to be lowered

Given the fact that 5 out of the 8 checked instances were labeled wrong by worker A, A_3 doubted worker A's reliability. To quickly examine the labels of the remaining images labeled as "tree sparrow" by both the model and worker A, A_3 hovered over the row name "tree sparrow" on the worker's confusion matrix to highlight these images in the instance visualization. As shown in Fig. 6(a), these images were projected in both areas of the two species. Similarly, the instances labeled as "house sparrow" by worker A were also projected evenly on the space. Accordingly, A_3 verified worker A as a random partial spammer between "house sparrow" and "tree sparrow." Then we utilized the two-step strategy (Sec. 5.3) to process worker A.

The spammer identification of worker A triggered an update in the MRG model that recommended several other workers needed to be investigated. As shown on the flow map (Fig. 1H), workers B (Fig. 1F) and C (Fig. 1G) were recommended but their spammer scores were not as high as other recommended workers. Being interested in these two workers, A_3 checked their confusion matrices and some labeled instances. Following a similar process to the analysis of worker

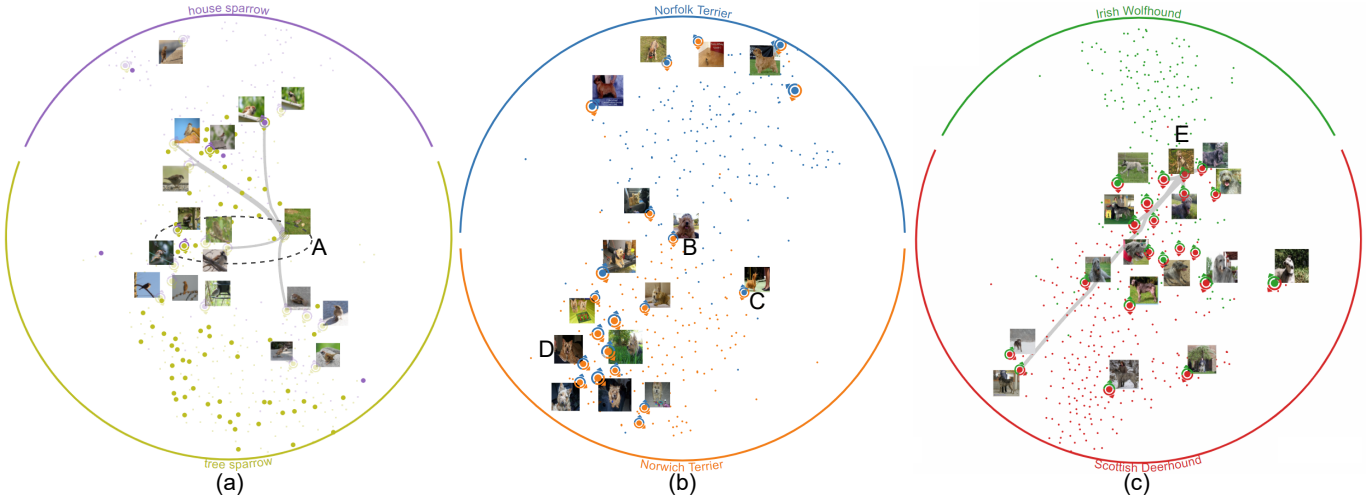

Fig. 6. Instance visualizations of the selected classes: (a) "house sparrow" and "tree sparrow"; (b) "Norfolk Terrier" and "Norwich Terrier"; (c) "Irish Wolfhound" and "Scottish Deerhound."

A, A_3 quickly confirmed that B and C were very likely to have the same spammer behavior as worker A. They tended to misidentify "tree sparrow" and "house sparrow" but their labeling results still matched the model's estimates to some extent. Therefore, A_3 verified them as random partial spammer with regards to these two species. After verifying 11 instances and 3 spammers, A_3 finished his first round of validation and propagated his verifications using the M^{3}V-based model. At that point, the labeling accuracy of the bird dataset had increased from **82.05%** to **83.15%**.

Detecting More Spammers After Propagation (T4, T5). After the first round of propagation, A_3 found that the spammer score increased a great deal for one unverified worker. A_3 checked this worker using a similar method as for worker A, and verified that s/he was also a random spammer. A_3 also noticed that the spammer score of worker A changed from 0.089 to 0.603. However, the spammer score was still not high given the fact that it was now verified as a random partial spammer for these two classes. Therefore, A_3 checked the uncertain instances recommended by the tool and also labeled by worker A, because doing so A_3 could very likely find more instances mislabeled by the M^{3}V-based model. Similarly, A_3 also checked some instance labels that had been suggested by the tool and labeled by other spammers. Following this strategy, A_3 verified 16 instances until the spammer scores of those verified spammers became higher. During the validation process, A_3 found two images that were hard to distinguish, as shown in Figs. 2(c-d), because they lacked the key features necessary to differentiate "house sparrow" and "tree sparrow." A_3 then asked the expert to identify the labels of these two images and validated them with the expert labels. Referring to the validation trail, the propagation also became stable, which indicated that there were not many instances that had been influenced by the changed labels. A_3 was now satisfied with the current validation between "house sparrow" and "tree sparrow." In total 4 spammers were identified and 33 instances were verified, and the accuracy improved to **84.45%**.

Validating Other Classes. After finishing the validation with the two sparrow-related classes, A_3 turned to other confusion pairs. Similar cases were found in which some spammers had matching labels with the model so that their spammer scores were low in the beginning. Similar to the exploration for the sparrow-related classes, A_3 validated 9 instances for eagle/falcon species, 8 for owls, 7 for sandpipers, and 3 for swans. Correspondingly, the accuracy improved to **86.10%, 86.30%, 87.50%, and 89.55%**. As shown in Table 1, compared with the M^{3}V-based model (84.50%), LapelInspect achieved a higher accuracy when validating the same number of instances (3.0% validation effort). The **validation effort** is measured by the proportion of the validated instances among all instances. Given the accuracy of **89.55%**, LabelInspect requires considerably less validation efforts (3.0%) than the M^{3}V-based model (more than 10.0%), which reduces **expert effort by 70.0%** [36]. This is due to the fact that LabelInspect helped A_1 and A_3 select more representative instances and spammers by leveraging the coordinated visualizations. For example, A_3 commented, "Worker inspection, instance verifications, and their mutual influence help enhance my validation effectiveness," while the M^{3}V-based model [36] only considers the instance label validation.

Table 1. Accuracy(%) of the M^{3}V-based model at different efforts.

Data \ Effort	1.0%	2.5%	3.0%	5.0%	7.5%	10.0%
Bird	82.65	83.90	84.50	87.45	88.55	89.05
Dog	88.88	90.00	90.50	92.00	92.65	93.00

6.2 Dog Image Validation

This case study is used to demonstrate that LabelInspect can also improve crowdsourced annotations with higher estimation accuracy. For this case study, we collaborated with A_2 and A_4. The dog dataset has four dog species labeled with AMT and the results estimated by the M^{3}V-based model have a relatively high accuracy (**88.13%**). However, validation is still desired by A_4 to meet the high accuracy requirement of his classification task.

Overview (T1). Starting with the confusion visualization, A_2 saw a clear block pattern that indicated the terrier-related classes and the hound-related classes were separable while there seemed to be confusion between "Norfolk Terrier" and "Norwich Terrier,' and "Irish Wolfhound" and "Scottish Deerhound." A_2 decided to start with the classes "Norfolk Terrier" and "Norwich Terrier."

Confirming Model Inferred Labels (T2, T3). The instance visualization (Fig. 6(b)) showed that these two species were separated well. Regarding the image snapshots with the uncertain glyphs, it seemed that most of the label estimates were correct. However, A_2 looked at the glyphs in the center and found that instance A (Fig. 6B) was labeled as "Norfolk Terrier" (blue) by many workers while the model estimated it to be "Norwich Terrier" (orange). Noticing this contradiction, A_2 clicked on the instance to check. The worker visualization showed that although fewer workers labeled it as "Norwich Terrier," they in general had high accuracy and low spammer score. A_2 found that this instance was recommended by the tool because many unreliable workers had mislabeled it and the model's estimate was reliable. Therefore, A_2 confirmed the label of instance A and two other instances like this (Fig. 6C and Fig. 6D). In addition, A_2 also verified some other recommended instances by MRG. For the terrier-related classes, 7 instances were validated and the accuracy was improved to **90.75%**.

Focusing on Instance Analysis (T1, T4). Next, A_4 selected the classes "Irish Wolfhound" and "Scottish Deerhound" for further validation. The instance visualization of these two types of hounds showed a

high level of confusion (Fig. 6(c)). The recommended instances were all estimated as "Scottish Deerhound" (red). However, A_4 spotted many label errors by looking at the image snapshots. A_4 speculated that the M^{3}V-based model was inclined to assign the label "Scottish Deerhound" (red) to the instances, and this also aligned with the fact that the red arc was much longer than the green arc. To investigate the underlying cause, A_4 focused on validating instances that were estimated to be "Scottish Deerhound" (red). A_4 started with instance B (Fig. 6E), which was mixed with many green instances. Checking the image, A_4 found that instance B should be "Irish Wolfhound" and then corrected it. The 5 most influenced instances, indicated by the flow map, were checked by A_4 consecutively and they were all mislabeled. A_4 propagated his validation after correcting these 6 labels. One instance moved a long way on the projected space and A_4 examined it closely. This instance was also labeled wrong and A_4 then assigned "Scottish Deerhound" to it. By repeating the above validation procedure, 13 instances were validated and the accuracy was improved to **93.00%**. In total, we validated 20 instances (validation efforts: 2.5%) to achieve an accuracy of **93.00%**. As shown in Table 1, to achieve the same accuracy, the M^{3}V-based model requires considerably more validation efforts (10.0%) than LabelInpsect. Thus, our tool reduce **75.0% expert effort**. A_4 believed that this accuracy was acceptable for his classification task, so he stopped the verification process.

7 Discussion

Our visual analytics method has presented an evidence that well designed interactive visualizations provide an effective data validation environment, which is consistent with the previous findings that in many cases, visual analytics methods work better than pure machine-learning-based methods [15, 40, 41, 44, 56]. For the bird dataset, the initial accuracy of the label estimates learned by the M^{3}V-based model was not high (82.05%). This led to a slight misjudgment where several spammers did not have high spammer scores. The first case study demonstrated that LabelInspect compensated for the M^{3}V-based model greatly and improved the labeling accuracy effectively by defining partial spammers and allowing the analyst to investigate both workers and instances. On the other hand, the dog dataset has relatively high labeling quality and the initial estimation accuracy was high (88.13%). For this type of dataset, the spammer score measures the worker's reliability quite well, and the analyst focused more on investigating and validating instances. The second case study demonstrated that iterative uncertain candidate selection and validation propagation effectively improved the data labeling accuracy even on top of a high quality dataset.

In addition to conducting two case studies, we also shared our system with three analysts from different research institutes. Overall, LabelInspect was well received by the analysts. They confirmed the usefulness of the tool for validating crowdsourced annotations and improving data quality. The visualization design was praised and one analyst who played with this system for about an hour commented, "I can quickly find that 'tree sparrow' and 'house sparrow' are badly separated," and he found the spammers with a quick glance. The flow map on the instance visualization hinted to the newly identified uncertain instances and most influenced instances. He liked the combination of the instance recommendation and the model propagation. The analyst commented, "Whenever I fail to find more to validate, I use the propagate button, which seems to work pretty well." Another analyst who spent over two weeks to tune the parameters of the M^{3}V-based model without the help of LabelInspect improved labeled accuracy from 82.05% to 89.05% commented, "LabelInspect is very effective and useful in helping me improve the data quality of crowdsourced annotations. It only took me two hours to improve the data quality to a satisfactory level, which usually takes me one or more weeks." One crowdsourcing expert explicitly said, "This is valuable work," and he liked the worker visualization a lot, especially the idea of validating spammers and indicating on which classes they are likely to be wrong. "Validating spammer workers is a remarkable function in this tool, and identifying partial spammers is very useful. It saves more effort than validating one image a time and provides more benefits."

Although LabelInpsect was positively received by the analysts, they suggested several directions to improve this tool.

Generalization. In our implementation, image data was taken as an example to illustrate the basic idea of interactive data validation. The analysts expressed the need to employ LabelInspect to validate other types of crowdsourced data. Our method can easily be extended to handle other types of data with only some minor changes of the visualization components. For example, for textual data, the snapshot in the instance visualization can be replaced with a keyword-based text summarization, and the detail view can display a multimedia-based summary generated by Document Cards [53], which includes the important keywords and images of a document. Other than crowdsourced data validation, the novel concept proposed in this paper can also be generalized to other usages. The constrained t-SNE projection is a visualization technique that can be used for generalized iterative analysis of uncertain training data, and the spammer score proposed in this paper together with the worker visualization can also be generalized to any crowdsourced labeling analysis. In addition, the method for integrating worker analysis and instance analysis is also able to generalize for multi-facet labeling tasks.

Scalability. The bird dataset has about 2,000 images and LabelInspect can respond to each interaction in real time. However, when the number of instances becomes large (e.g., tens of thousands of instances), the instance projection and the model propagation will suffer from increased computational cost. Potential solutions are to employ a sampling technique (e.g., blue noise sampling [11, 39]) in the t-SNE based projection, and to develop a scalable label propagation algorithm.

The instance visualization will become cluttered when the number of instances increase, and it is difficult to detect useful patterns from the visualization. One possible solution is to combine a density-based visualization [35, 60] with a t-SNE projection to reveal overall patterns, important instances, and outliers.

Insufficient handling of wrong validation. Currently, our method regards the analysts' validation as ground truth. However, this may not always be correct. A wrong validation can lead to a decline in accuracy. One possible solution to avoid accepting wrong validation is to examine the gap between the model-centered estimation and human-centered analysis. If the gap is large, we could ask the analyst to re-check his/her validation based on a visual explanation that illustrates why the model recommended a different prediction. Other more effective solutions to this problem are still open to investigation.

8 Conclusion

In this paper, we have developed LabelInspect, a visual analytics tool to support crowdsourced data validation for preparing labeled training data. LabelInspect incorporates learning models and visualizations and enables mutual influencing validation from both the instance-level and worker-level data. New techniques proposed include a constraint t-SNE projection for instance visualization, a block-based spammer score calculation, and an integrated system that incorporates expert's verifications into an iterative and progressive validation process. We conducted two case studies to demonstrate the effectiveness of LabelInspect and the results indicate that label accuracy has been improved successfully.

Although LabelInspect has been shown to be effective, further work on the following aspects is still promising. First, addressing the above mentioned limitations in terms of scalability and expert trust would be critical for prompting this tool to a wide usage. Second, to strengthen the analysis of worker behavior, reputation-based worker/instance filtering needs to be further studied. Third, it might be beneficial to allow multiple experts to work together on validating the same datasets for a more accurate labeled dataset.

Acknowledgments

This research was funded by National Key R&D Program of China (No. SQ2018YFB100002), the National Natural Science Foundation of China (No.s 61761136020, 61672308), and Microsoft Research Asia. The authors would like to thank Liwang Zhou for implementing part of the learning-from-crowds algorithm and the anonymous reviewers for their valuable comments. B. Wang is the corresponding author.

REFERENCES

[1] C. Arbesser, F. Spechtenhauser, T. Mühlbacher, and H. Piringer. Visplause: Visual data quality assessment of many time series using plausibility checks. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):641–650, 2017.

[2] Z. Bar-Joseph, D. K. Gifford, and T. S. Jaakkola. Fast optimal leaf ordering for hierarchical clustering. *Bioinformatics*, 17(suppl_1):S22–S29, 2001.

[3] M. Behrisch, B. Bach, N. Henry Riche, T. Schreck, and J.-D. Fekete. Matrix reordering methods for table and network visualization. *Computer Graphics Forum*, 35(3):693–716, 2016.

[4] M. Behrisch, F. Korkmaz, L. Shao, and T. Schreck. Feedback-driven interactive exploration of large multidimensional data supported by visual classifier. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pages 43–52, 2014.

[5] J. Bernard, M. Hutter, M. Zeppelzauer, D. Fellner, and M. Sedlmair. Comparing visual-interactive labeling with active learning: An experimental study. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):298–308, 2018.

[6] J. Bernard, D. Sessler, A. Bannach, T. May, and J. Kohlhammer. A visual active learning system for the assessment of patient well-being in prostate cancer research. In *The Workshop on Visual Analytics in Healthcare*, pages 1:1–1:8, 2015.

[7] T. Blascheck, F. Beck, S. Baltes, T. Ertl, and D. Weiskopf. Visual analysis and coding of data-rich user behavior. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pages 141–150, 2016.

[8] S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1-7):107–117, 1998.

[9] P. Bruneau and B. Otjacques. An interactive, example-based, visual clustering system. In *Proceedings of the International Conference on Information Visualisation*, pages 168–173, 2013.

[10] K. Buchin, B. Speckmann, and K. Verbeek. Flow map layout via spiral trees. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2536–2544, 2011.

[11] H. Chen, W. Chen, H. Mei, Z. Liu, K. Zhou, W. Chen, W. Gu, and K.-L. Ma. Visual abstraction and exploration of multi-class scatterplots. *IEEE Transactions on Visualization and Computer Graphics*, 20(12):1683–1692, 2014.

[12] F. Chollet et al. Keras. `https://keras.io/`, 2018. Last accessed 2018-03-31.

[13] J. Choo, C. Lee, C. K. Reddy, and H. Park. Utopian: User-driven topic modeling based on interactive nonnegative matrix factorization. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):1992–2001, 2013.

[14] J. Choo and S. Liu. Visual analytics for explainable deep learning. *IEEE Computer Graphics and Applications*, 38(4):84–92, 2018.

[15] W. Cui, S. Liu, L. Tan, C. Shi, Y. Song, Z. Gao, H. Qu, and X. Tong. Textflow: Towards better understanding of evolving topics in text. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2412–2421, 2011.

[16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[17] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. DeCAF: A deep convolutional activation feature for generic visual recognition. In *Proceedings of the International Conference on Machine Learning*, pages 647–655, 2014.

[18] Y. Duan, Z. Chen, F. Wei, M. Zhou, and H.-Y. Shum. Twitter topic summarization by ranking tweets using social influence and content quality. In *Proceedings of the International Conference on Computational Linguistics*, pages 763–780, 2012.

[19] C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[20] A. Endert, W. Ribarsky, C. Turkay, B. Wong, I. Nabney, I. D. Blanco, and F. Rossi. The state of the art in integrating machine learning into visual analytics. *Computer Graphics Forum*, 36(8):458–486, 2017.

[21] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 580–587, 2014.

[22] M. C. Hao, H. Janetzko, S. Mittelstädt, W. Hill, U. Dayal, D. A. Keim, M. Marwah, and R. K. Sharma. A visual analytics approach for peak-preserving prediction of large seasonal time series. *Computer Graphics Forum*, 30(3):691–700, 2011.

[23] F. Heimerl, S. Koch, H. Bosch, and T. Ertl. Visual classifier training for text document retrieval. *IEEE Transactions on Visualization and Computer Graphics*, 18(12):2839–2848, 2012.

[24] J. Himberg, K. Korpiaho, H. Mannila, J. Tikanmaki, and H. T. Toivonen. Time series segmentation for context recognition in mobile devices. In *Proceedings of the IEEE International Conference on Data Mining*, pages 203–210, 2001.

[25] B. Höferlin, R. Netzel, M. Höferlin, D. Weiskopf, and G. Heidemann. Inter-active learning of ad-hoc classifiers for video visual analytics. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pages 23–32, 2012.

[26] D. Holten and J. J. Van Wijk. Force-directed edge bundling for graph visualization. *Computer Graphics Forum*, 28(3):983–990, 2009.

[27] N. Q. V. Hung, D. C. Thang, M. Weidlich, and K. Aberer. Minimizing efforts in validating crowd answers. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 999–1014, 2015.

[28] S. Jagabathula, L. Subramanian, and A. Venkataraman. Reputation-based worker filtering in crowdsourcing. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 2492–2500, 2014.

[29] G. Jeh and J. Widom. SimRank: A measure of structural-context similarity. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 538–543, 2002.

[30] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer. Wrangler: Interactive visual specification of data transformation scripts. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 3363–3372, 2011.

[31] S. Kandel, R. Parikh, A. Paepcke, J. M. Hellerstein, and J. Heer. Profiler: Integrated statistical analysis and visualization for data quality assessment. In *Proceedings of the International Conference on Advanced Visual Interfaces*, pages 547–554, 2012.

[32] G. Kazai, J. Kamps, and N. Milic-Frayling. Worker types and personality traits in crowdsourcing relevance labels. In *Proceedings of the ACM International Conference on Information and Knowledge Management*, pages 1941–1944, 2011.

[33] M. Kim, K. Kang, D. Park, J. Choo, and N. Elmqvist. TopicLens: Efficient multi-level visual topic exploration of large-scale document collections. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):151–160, 2017.

[34] J. Krause, A. Perer, and H. Stavropoulos. Supporting iterative cohort construction with visual temporal queries. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):91–100, 2016.

[35] H. Liao, Y. Wu, L. Chen, and W. Chen. Cluster-based visual abstraction for multivariate scatterplots. *IEEE Transactions on Visualization and Computer Graphics (accepted)*, 2018.

[36] M. Liu, L. Jiang, J. Liu, X. Wang, J. Zhu, and S. Liu. Improving learning-from-crowds through expert validation. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 2329–2336, 2017.

[37] M. Liu, S. Liu, X. Zhu, Q. Liao, F. Wei, and S. Pan. An uncertainty-aware approach for exploratory microblog retrieval. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):250–259, 2016.

[38] M. Liu, J. Shi, K. Cao, J. Zhu, and S. Liu. Analyzing the training processes of deep generative models. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):77–87, 2018.

[39] M. Liu, J. Shi, Z. Li, C. Li, J. Zhu, and S. Liu. Towards better analysis of deep convolutional neural networks. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):91–100, 2017.

[40] S. Liu, W. Cui, Y. Wu, and M. Liu. A survey on information visualization: recent advances and challenges. *The Visual Computer*, 30(12):1373–1393, 2014.

[41] S. Liu, X. Wang, C. Collins, W. Dou, F. Ouyang, M. El-Assady, L. Jiang, and D. Keim. Bridging text visualization and mining: A task-driven survey. *IEEE Transactions on Visualization and Computer Graphics (accepted)*, 2018.

[42] S. Liu, X. Wang, M. Liu, and J. Zhu. Towards better analysis of machine learning models: A visual analytics perspective. *Visual Informatics*, 1(1):48–56, 2017.

[43] S. Liu, J. Xiao, J. Liu, X. Wang, J. Wu, and J. Zhu. Visual diagnosis of tree boosting methods. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):163–173, 2018.

[44] S. Liu, M. X. Zhou, S. Pan, Y. Song, W. Qian, W. Cai, and X. Lian. Tiara:

Interactive, topic-based visual text summarization and analysis. *ACM Transactions on Intelligent Systems and Technology*, 3(2):25, 2012.

[45] Y. Lu, H. Wang, S. Landis, and R. Maciejewski. A visual analytics framework for identifying topic drivers in media events. *IEEE Transactions on Visualization and Computer Graphics (accepted)*, 2018.

[46] L. v. d. Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.

[47] R. Maciejewski, A. Pattath, S. Ko, R. Hafen, W. S. Cleveland, and D. S. Ebert. Automated box-cox transformations for improved visual encoding. *IEEE Transactions on Visualization and Computer Graphics*, 19(1):130–140, 2013.

[48] Y. Meng, H. Zhang, M. Liu, and S. Liu. Clutter-aware label layout. In *Proceedings of the IEEE Pacific Visualization Symposium*, pages 207–214, 2015.

[49] J. G. S. Paiva, W. R. Schwartz, H. Pedrini, and R. Minghim. An approach to supporting incremental visual data classification. *IEEE Transactions on Visualization and Computer Graphics*, 21(1):4–17, 2015.

[50] J. H. Park, S. Nadeem, S. Mirhosseini, and A. Kaufman. C^2A: Crowd consensus analytics for virtual colonoscopy. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pages 21–30, 2016.

[51] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[52] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *Computing Research Repository*, abs/1409.1556, 2014.

[53] H. Strobelt, D. Oelke, C. Rohrdantz, A. Stoffel, D. A. Keim, and O. Deussen. Document cards: A top trumps visualization for documents. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):1145–1152, 2009.

[54] J. Sun, F. Wang, J. Hu, and S. Edabollahi. Supervised patient similarity measure of heterogeneous patient records. *ACM SIGKDD Explorations Newsletter*, 14(1):16–24, 2012.

[55] J. Talbot, B. Lee, A. Kapoor, and D. S. Tan. EnsembleMatrix: Interactive visualization to support machine learning with multiple classifiers. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 1283–1292, 2009.

[56] G. K. Tam, V. Kothari, and M. Chen. An analysis of machine- and human-analytics in classification. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):71–80, 2017.

[57] T. Tian and J. Zhu. Max-margin majority voting for learning from crowds. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 1621–1629, 2015.

[58] P. Wais, S. Lingamneni, D. Cook, J. Fennell, B. Goldenberg, D. Lubarov, D. Marin, and H. Simons. Towards building a high-quality workforce with mechanical turk. In *NIPS Workshop on Computational Social Science and the Wisdom of Crowds*, pages 1–5, 2010.

[59] E. Wall, S. Das, R. Chawla, B. Kalidindi, E. T. Brown, and A. Endert. Podium: Ranking data using mixed-initiative visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):288–297, 2018.

[60] X. Wang, S. Liu, J. Liu, J. Chen, J. Zhu, and B. Guo. TopicPanorama: A full picture of relevant topics. *IEEE Transactions on Visualization and Computer Graphics*, 22(12):2508–2521, 2016.

[61] L. Wilkinson. Visualizing big data outliers through distributed aggregation. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):256–266, 2018.

[62] W. Willett, S. Ginosar, A. Steinitz, B. Hartmann, and M. Agrawala. Identifying redundancy and exposing provenance in crowdsourced data analysis. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2198–2206, 2013.

[63] J. Xia, W. Chen, Y. Hou, W. Hu, X. Huang, and D. S. Ebertk. Dimscanner: A relation-based visual exploration approach towards data dimension inspection. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pages 81–90, 2016.

[64] M. D. Zeiler, G. W. Taylor, and R. Fergus. Adaptive deconvolutional networks for mid and high level feature learning. In *Proceedings of the International Conference on Computer Vision*, pages 2018–2025, 2011.

[65] D. Zhou, S. Basu, Y. Mao, and J. C. Platt. Learning from the wisdom of crowds by minimax entropy. In *Proceedings of the Advances in Neural Information Processing Systems*, pages 2195–2203, 2012.